

Session Review — 2026-07-28

What this covers. The work after `docs/ARCHITECTURE-REVIEW-2026-07-27.md`. Every number here came from running something today. Where a document and a measurement disagreed, the measurement won.

For review. Section 7 lists the files where your judgement beats code review, and why.

1. Executive summary

Four claims in this project asserted more than their own numbers supported. Three are now withdrawn and the fourth is gated. In every case the code compared a **point estimate** to a baseline and ignored its own uncertainty range.

claim	was	is
CHOMP picks winning brings	"the winning direction"	51% , range 49–53%. A coin
PORY is a real value net	"beats coin and material"	its range contains both. A material counter
Trick Room > Hyper Offense > Sand	"a real non-transitive cycle"	two of three legs are coin flips , on 23 games each
CHOMP inputs are clean	assumed	one was built on the raw store

The fifth finding is a null that cost nothing to get: giving MAG back the team-sheet information it was trained on changed its win rate **not at all**.

The single most useful measurement of the day is that CHOMP, rebuilt on fully clean data, still loses to a naive baseline. That answers "is CHOMP worth keeping" with evidence rather than opinion.

What did not survive scrutiny is not the same as what is broken. Nothing regressed today. Four things stopped claiming to be better than they are.

2. Findings, ranked by blast radius

F1 — A CHOMP input was built on the raw store, and the guard said nothing

`engine/selftest.js` checks that every file naming the ladder store either filters it or declares why not. It looked for the text `load_games`. Three files **defined their own function with that name** that read the store line by line with no filter at all, so they passed the check by naming a function:

- `engine/xatu_context.py` — builds `data/xatu-context-sets.json`, **which CHOMP consumes**
- `engine/xatu_belief.py`
- `engine/train_value.py`

A false negative is the one error this check must never make. It produced exactly the outcome the rule exists to prevent: a CHOMP input derived from a population that is roughly **87% bot games, forfeits, partial brings and stubs**, with the guard reporting no offence.

This also corrects yesterday's correction. The offender count was taken from 17 to 12 by recognising three files that only *mention* the path. That was right and incomplete — the same pass should have found these three. **The true count was 15.** Over-counting is noise. Under-counting is a clean bill of health for contaminated data, and it is the worse direction to be wrong in.

Fixed. The guard is now structural: a loader name counts as evidence of filtering only if the file did not define that loader itself. Stripping the definition and re-testing was the first attempt and does not work, because the file goes on to *call* its own loader.

F2 — CHOMP does not survive clean data

Regenerated in dependency order — roles, then context, then CHOMP-EV — against **2,653 usable games of 20,387 collected**. Tested on 2,603 held-out games, up from 2,200.

How often each approach picks the winner, where **50% is a coin flip**:

approach	gets it right
naive "bring your four most-used"	best of the six
CHOMP alignment	slightly worse than naive
CHOMP + team context	slightly worse again
CHOMP + belief weighting	worse again
a coin	50%
player Elo rating	worse than a coin

Every CHOMP variant's uncertainty range **contains the coin**. The naive baseline beats all three. And each added layer of sophistication scores *worse* than the one before, which agrees with a null result this project had already found — better beliefs did not improve the bring decision.

The bring effect barely moved after cleaning: **51.3% → 51.3%**, range 49.4–53.2%, still spanning 50%.

Stated plainly: CHOMP has no demonstrated edge over bringing your four most-used Pokémon.

Two things that *did* survive, and deserve saying:

- **The roles null result holds on 2.5× the data.** Role-level winner prediction still ties a coin.
- **Team context is a small real effect.** It beats the usage prior on first-move prediction by a margin that clears zero — the only positive result of the day.

F3 — The cycle detector could not fail to find a cycle

`top_cycle()` searched **every ordered triple** — 990 of them for 11 playstyles — kept whichever had the strongest weakest leg, and asserted in its own docstring that "*A positive strength means a real cycle exists.*" No sample-size floor. No significance test. On a sparse matrix it therefore always returns something positive. `tests/test-slowking.py` then **required** a cycle to be reported, locking the artefact in place.

The cycle it reported, measured:

leg	wins	games	range	verdict
Trick Room beats Hyper Offense	65%	23	45–81%	could be a coin
Hyper Offense beats Sand	57%	23	37–74%	could be a coin
Sand beats Trick Room	71%	24	51–85%	clears, barely

A three-way cycle needs all three legs. Two of them are indistinguishable from chance, on 23 games each.

Requiring every leg to both clear 50% **and** rest on 50+ games leaves **no cycle at all** in that matrix. And across 990 candidate triples at the usual threshold you would expect roughly **17 spurious cycles by chance** even in a perfectly balanced metagame. Finding one is the expected result of noise.

Fixed. The strongest triple is still reported as a descriptive summary, but now carries the games behind each leg, the number of triples searched, and an explicit `supported` flag that is false unless every leg is both a win and adequately sampled.

What this does and does not change. It kills the *explanation* for why mixing matters. It does not rescue greedy: you do not need a cycle to be exploitable, only an opponent able to answer what you commit to. That still holds, measured two ways — a fixed preview pick loses about **8 percentage points** to its best counter, and in battle a counter-policy built by crude search beats MAG **63%** of the time.

F4 — Two more verdicts gated on a point estimate

PORY. Its verdict compared point estimates and ignored the interval, so it printed *"a real, calibrated value net."* Its own range **contains** both the coin and the two-feature material heuristic.

Independent corroboration that the cautious reading is right — from a network trained on **1.3 million positions**:

model	gets the winner right
a coin	50%
alive-count alone — one number	60%
alive-count + HP — two numbers	63%
the neural network on material	63%
the network on the <i>rich</i> feature set	62%

The entire network buys under half a point over counting bodies and HP, and **every richer feature set scores worse than material-only**. PORY is a material counter. That is why a +2 setup move scores as worthless to it: nothing died and no HP moved.

There is also an undiagnosed warning sign — the network scored substantially better on validation than on test. Something differs between those splits and nobody has looked.

Fixed. Both verdicts now require the interval to clear the baseline.

F5 — The playstyle classifier discards a team's identity

`engine/playstyle.js` is a **first-match-wins cascade** of 11 rules, and Trick Room is checked **second**, before Setup, Stall, and every weather:

```

if (hasMove(PERISH_MOVE) ...) return 'PerishTrap';
if (hasMove(TR_MOVE) ...)    return 'TrickRoom';    // 2nd of 11
...
if (has(SUN))                return 'Sun';          // 7th

```

So a Sun team carrying one Trick Room is labelled **Trick Room, not Sun**. That is very likely why Trick Room is the largest class at **26% of all teams**.

The original objection raised in review — that a team carrying Trick Room might be carrying it to *undo* the opponent's — is a fair point that partly resolves itself, since flipping an opponent's Tailwind is still playing Trick Room. The cascade is the more serious defect, and it needs no interpretation to see.

Not fixed. The ordering encodes a judgement about which signal dominates, and that judgement is yours, not mine. It is listed in §7.

F6 — The equilibrium assigns real weight to eleven games

The Nash mixture over playstyles currently reads:

*Sand 0.50, Tailwind Offense 0.27, Hyper Offense 0.06, Setup 0.05, **Stall 0.05**, Perish Trap 0.05*

Stall has 11 observations. The solver cannot distinguish an unmeasured cell from a real one, so it allocates 5% of the strategy to noise.

The class sizes, out of 6,620 team-slots:

playstyle	slots	share	usable?
Trick Room	1,740	26%	yes — but see F5
Sun	1,417	21%	yes
Setup	1,063	16%	yes
Tailwind Offense	797	12%	yes
Rain	625	9%	yes
Sand	287	4%	marginal
Hyper Offense	261	4%	marginal
Perish Trap	220	3%	marginal
Snow	134	2%	too thin
FakeOut Balance	65	1%	too thin
Stall	11	0.2%	unusable

The retracted cycle was built from Sand, Hyper Offense and Trick Room — two of them in the marginal band. The bottom three should not be in a matrix yet, and their presence is what produces cells like "Tailwind Offense beats FakeOut Balance, 100% of 4 games" being reported as decisive.

Not fixed — dropping a class is a judgement call. §7.

F7 — Restoring MAG's sheet information changed nothing

Yesterday's head-to-head runs were launched without a format flag, so they defaulted to **closed** sheets. `mew.js` warns about exactly this in its own comments: several of MAG's features read the team sheet, and testing them in a closed-sheet game measures them switched off. MAG is *fitted* on open sheets and was being *graded* without them.

Re-run on the format that forces open sheets:

regime	decisive pairs	range
closed sheets	93.2%	92.3–94.1%
open sheets	92.8%	91.9–93.7%

Overlapping. **No measurable difference.** Yesterday's figures were not wrong, only measured in the wrong regime — but a whole feature family now looks inert.

Honest limit: the opponent was a random-clicking bot, whose behaviour is unpredictable regardless of what you know about its team. Sheet information may matter against a real player. Untested.

3. Where the models actually stand

model	status
MAG	Beats random 93% . Beats its own predecessor 55% — real but modest. Never measured against a human. Popularity helps it both at predicting people and at winning
CHOMP	No demonstrated edge over the naive baseline. Does not beat a coin
PORY	A material counter. Two numbers match its whole network
Roles	Ties a coin. Confirmed on 2.5× the data
Team context	Small real effect — the only positive of the day
Preview Nash	Less exploitable than greedy, but the interval barely clears zero and the mixture weights an 11-game class

What MAG actually is

Nine of its ten strongest signals are **negative**, and every one means *this move cannot work right now* — Encore into something that has not moved, a move their ability blocks, a type they are immune to, something already in place. The type-effectiveness signals are about a third as strong.

MAG is a blunder detector, not a strategist. That explains its whole results profile: it crushes an opponent that blunders constantly and barely edges one that already does not. It also explains why taking its top pick instead of rolling a weighted dice was worth more than every feature added.

What it structurally cannot do

There is **no lookahead anywhere** — no rollouts, no search, no depth. MAG scores this turn's options and stops. It knows "a move boosts me" and, separately and much more strongly, "I am currently boosted,

so swing" — but it never connects them. It cannot reason that spending this turn on a setup move buys a knockout next turn, because there is no next turn in its world.

CLAUDE.md already records this: *"Setup matters — Speed Boost + Swords Dance sweepers were undervalued by turn-1 damage alone."* The note went in. The capability did not. It is the clearest case in the project of a correction recorded rather than fixed.

One related dead feature: "their defence is lowered" has essentially **zero** weight with a range spanning zero. Across 82,836 human decisions the model could not detect that lowering an opponent's defence changes what people do.

4. Numbers corrected

figure	was	is
CHOMP bring effect	"CI clear of 0.5 — the winning direction"	51% , range 49–53%. Not significant
CHOMP vs a coin	not stated plainly	does not beat a coin
CHOMP vs naive baseline	never compared	loses to it
PORY verdict	"a real, calibrated value net"	range contains both baselines
PORY vs two features	implied superior	matched by two numbers
The playstyle cycle	"a real non-transitive cycle"	two of three legs are coin flips on 23 games
Raw-store offenders	17, then 12	15
CHOMP eval size	2,200 games	2,603 games
MAG head-to-head	93.2% closed sheets	92.8% open sheets — no difference
Roles null	ties a coin on 1,061 games	holds on 2,653

5. Causal claims audit

claim	verdict	evidence
A fixed pick is exploitable	VERIFIED	preview: loses ~8 points to its best counter, interval clears zero. In battle: a counter beats MAG 63%, mirror control 47%
A rock-paper-scissors cycle is why mixing matters	DISPROVEN as stated	two of three legs are coin flips; no cycle survives a 50-game floor; ~17 expected by chance across 990 triples
Real matchup structure exists	VERIFIED, but different	10 of 66 cells clear 50% on 50+ games, and nearly all involve Sun — which has 1,417 games behind it. Structure is real; it is not a triangle
PORY is a real value net	DISPROVEN	range contains both baselines; two features match the network
<code>pokemon-roles.json</code> was unsafe	DISPROVEN	<code>roles.py</code> has always filtered. The handoff was wrong. It was stale , not unsafe
<code>xatu-context-sets.json</code> was unsafe	VERIFIED	read the raw store with no filter, hidden from the guard by a local function name
Cleaning CHOMP's inputs would rescue it	DISPROVEN	51.3% before, 51.3% after
MAG's sheet features earn their keep	UNSUPPORTED	open vs closed sheets: no measurable difference against random. Untested against a real player
MAG is a blunder detector, not a strategist	CONSISTENT	9 of its 10 strongest weights are "this cannot work". Structural reading of the fitted model, not a separate experiment

6. What I could not do

- **Measure the classifier's revelation bias.** I tried twice and both attempts were invalid — the first counted both sides' moves, the second used the forced-open-sheet corpus, which by definition has no revelation bias. Proving it needs ground truth in games where the truth is hidden. The cascade defect (F5) stands on the code alone and needs no measurement.
- **Fix the classifier or drop the thin playstyles.** Both encode domain judgements that are yours.
- **Measure CHOMP's exploitability directly.** It needs a way to force a specific bring, which `mew.js` does not have — brings are sampled from priors. Scope: force-bring support, a new script to solve the 15×15 grid per team pair, and roughly 16,000 games. Worth doing, and expect a coin given that the clean matchup matrix has almost no decisive cells.
- **Test MAG against a human.** Needs you, or a decision about laddering publicly.
- **Retrain PORY on clean data**, or diagnose its validation/test gap.
- **Re-run the three older head-to-head cells** quoted from the handoff. They predate today's fixes.

7. For your review — where your judgement beats code review

Everything caught in review today shares a shape: **a human judgement encoded as a list or a threshold**. I can verify those are applied consistently. I cannot verify they are *right*. That distinction is the whole value of your reading these.

file	the judgement	the question
engine/playstyle.js	11-rule cascade, first match wins	Is Trick Room really the 2nd most defining signal, ahead of every weather?
same	STALL — 7 species	Right list?
same	SETUP_MOVES — 11 moves	Is Agility setup? Iron Defense?
same	slowLeaning() — 8 species	Right list?
same	countAtk >= 3	Why three?
engine/roles.py	52 roles; PRESENT_AT = 0.50	Are these the right roles, and is half the right bar?
data/quality-filter.json	50 games/1 team = bot; 3-turn floor; require all four brought	Is losing 487 games to the bring rule worth it?
engine/board.js FEATURES	47 features	What mechanic is missing? You found Encore and Prankster this way
engine/set_priors.js	FAMILY_OF — which moves compete for a slot	Which pairs are missing besides Protect/Detect?
engine/war.py	MIN_GAMES = 30	Enough appearances to rate a species?

Four findings today came from exactly this kind of reading, and none would have surfaced from code review: that a fixed pick must be exploitable; that setup moves need lookahead; that a team with Trick Room is not a Trick Room team; and that the open-sheet data is mostly an external archive rather than our own.

8. The strategic picture

You are data-poor on people and data-infinite on the game.

Open-sheet games — the only kind MAG can be fitted on, because you must know what a player *could* have clicked — total roughly **3,600 clean games**, and **58% of them come from an external archive** rather than your own scrape. Your own forced-open-sheet ladder is the smallest of the three sources and the only tap you control.

Everything thin today traces to that one fact:

- the cycle died on 23-game cells
- CHOMP cannot separate from a coin
- roles cannot beat a coin
- MAG beats its predecessor by 55%

Those are not four problems. They are **one sample size** appearing four times. A few thousand games can detect large effects, and there do not appear to be large effects here.

The division the project already articulated, and should now act on:

questions about people — capped	questions about the game — unlimited
what will they bring, what will they click	is this move ever correct here
does rating predict	does lookahead beat no lookahead
stop asking small ones — hunting them manufactures findings, which happened three times today	how exploitable is this policy
	does a better value function win more

Everything in the right column is answered by self-play, and self-play is free — 9,646 games in six minutes.

The architecture the evidence points at

Three problems keep appearing, and they have one fix:

1. **No lookahead** — cannot value a setup move
2. **A value function that only counts material** — scores a +2 boost as nothing
3. **A deterministic policy** — punished for 63% by a counter built to beat it

A policy that proposes candidates, a search that rolls them forward, and a value function that scores the result is the standard answer, and **you already have two of the three parts**: MAG is a good pruner precisely because it is a blunder filter, and `champions_sim` simulates a turn in about 50 milliseconds.

Cost of the search, from your own throughput: the naive all-against-all is ~225 rollouts, about 11 seconds — too slow. Narrowed to MAG's top 5 options against their 3 likeliest, sampled 3 times, it is **45 rollouts, roughly 2 seconds**. Affordable in a live game.

The missing third is a value function that can tell that being at +2 with your sweeper alive is a better position than the material count admits. That is the thing worth building, it needs no human data, and it fixes the lookahead problem and the exploitability problem at the same time.